CORE CONCEPTS

Explore the .claude directory

Where Claude Code reads CLAUDE.md, settings.json, hooks, skills, commands, subagents, workflows, rules, and auto memory. Explore the .claude directory in your project and ~/.claude in your home directory.

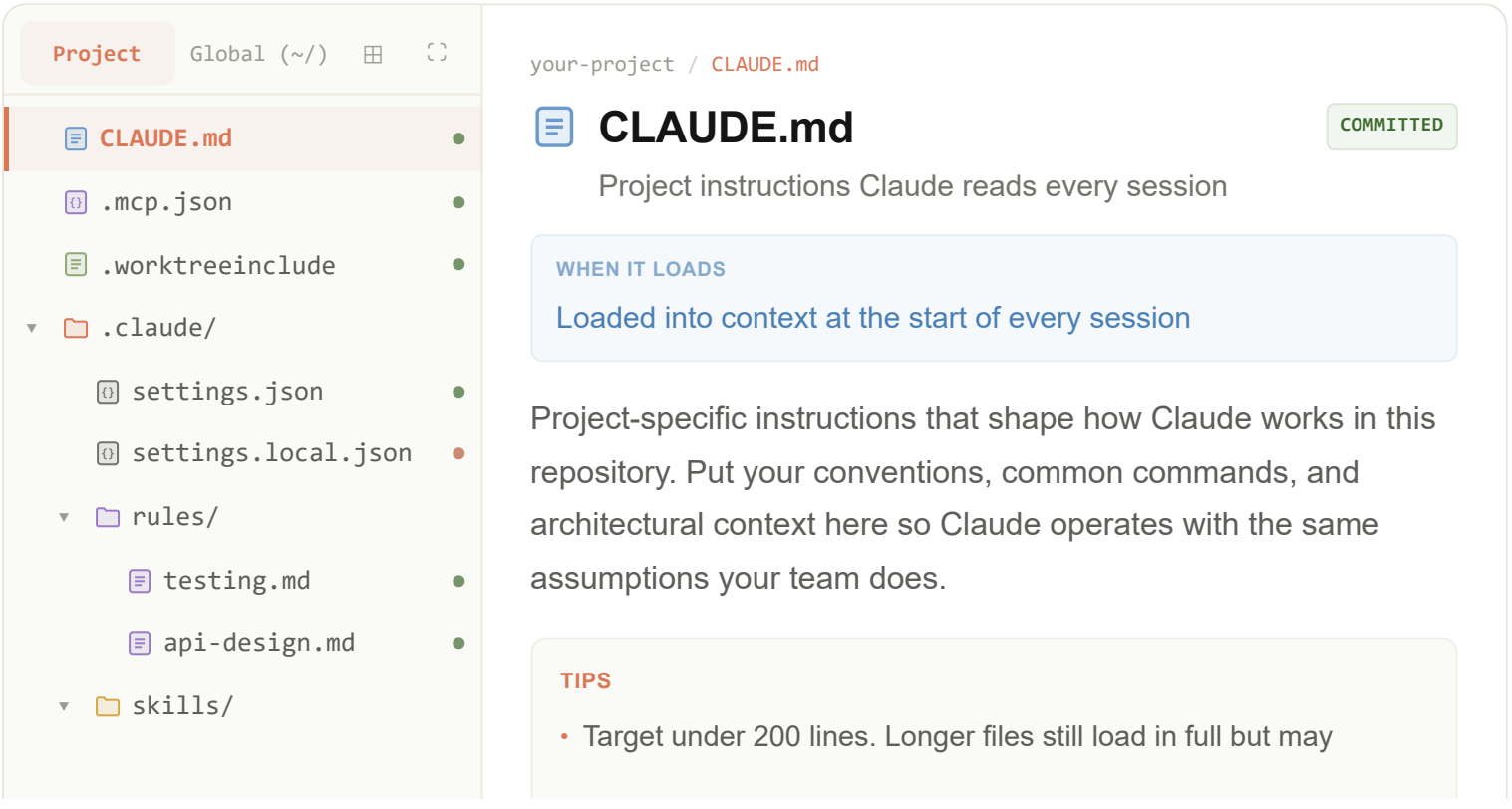
Claude Code reads instructions, settings, skills, subagents, and memory from your project directory and from `~/.claude` in your home directory. Commit project files to git to share them with your team; files in `~/.claude` are personal configuration that applies across all your projects.

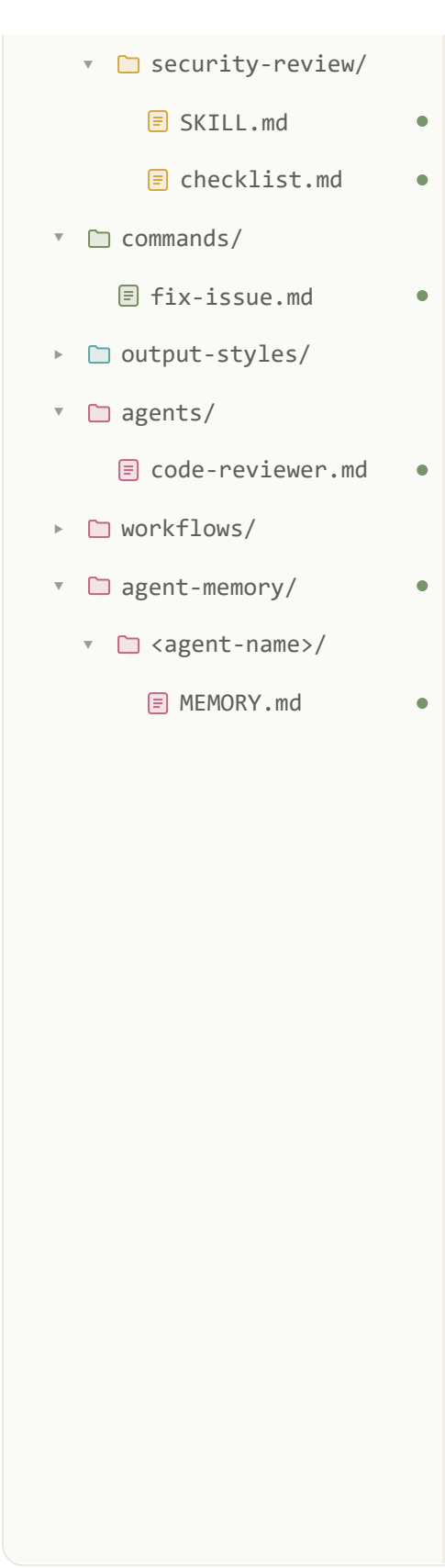
On Windows, `~/.claude` resolves to `%USERPROFILE%\.claude`. If you set `CLAUDE_CONFIG_DIR`, every `~/.claude` path on this page lives under that directory instead.

Most users only edit `CLAUDE.md` and `settings.json`. The rest of the directory is optional: add skills, rules, or subagents as you need them.

Explore the directory

Click files in the tree to see what each one does, when it loads, and an example.





reduce adherence

- CLAUDE.md loads into every session. If something only matters for specific tasks, move it to a **skill** or a path-scoped **rule** so it loads only when needed
- List the commands you run most, like build, test, and format, so Claude knows them without you spelling them out each time
- Run `/memory` to open and edit CLAUDE.md from within a session
- Also works at `.claude/CLAUDE.md` if you prefer to keep the project root clean

This example is for a TypeScript and React project. It lists the build and test commands, the framework conventions Claude should follow, and project-specific rules like export style and file layout.

CLAUDE.md

```
# Project conventions

## Commands
- Build: `npm run build`
- Test: `npm test`
- Lint: `npm run lint`

## Stack
- TypeScript with strict mode
- React 19, functional components only

## Rules
- Named exports, never default exports
- Tests live next to source: `foo.ts` -> `foo.test.ts`
- All API routes return `{ data, error }` shape
```

Full docs →

What's not shown

The explorer covers files you author and edit. A few related files live elsewhere:

File	Location	Purpose
`managed-settings.json`	System-level, varies by OS	Enterprise-enforced settings that you can't override. See **server-managed settings**.

File	Location	Purpose
`CLAUDE.local.md`	Project root	Your private preferences for this project, loaded alongside CLAUDE.md. Create it manually and add it to `.gitignore`.
Installed plugins	`~/.claude/plugins`	Cloned marketplaces, installed plugin versions, and per-plugin data, managed by `claude plugin` commands. Orphaned versions are deleted 7 days after a plugin update or uninstall. See **plugin caching**.

`~/.claude` also holds data Claude Code writes as you work: transcripts, prompt history, file snapshots, caches, and logs. See **application data** below.

Choose the right file

Different kinds of customization live in different files. Use this table to find where a change belongs.

You want to	Edit	Scope	Reference
Give Claude project context and conventions	`CLAUDE.md`	project or global	**Memory**
Allow or block specific tool calls	`settings.json` `permissions` or `hooks`	project or global	**Permissions**, **Hooks**
Run a script before or after tool calls	`settings.json` `hooks`	project or global	**Hooks**
Set environment variables for the session	`settings.json` `env`	project or global	**Settings**
Keep personal overrides out of git	`settings.local.json`	project only	**Settings scopes**
Add a prompt or capability you invoke with `/name`	`skills/<name>/SKILL.md`	project or global	**Skills**
Define a specialized subagent with its own tools	`agents/*.md`	project or global	**Subagents**
Orchestrate many subagents from a script	`workflows/*.js`	project or global	**Dynamic workflows**
Connect external tools over MCP	`.mcp.json`	project only	**MCP**
Change how Claude formats responses	`output-styles/*.md`	project or global	**Output styles**

File reference

This table lists every file the explorer covers. Project-scope files live in your repo under `.claude/` (or at the root for `CLAUDE.md`, `.mcp.json`, and `.worktreeinclude`). Global-scope files live in `~/.claude/` and apply across all projects.

> Several things can override what you put in these files:
>
> - **Managed settings** deployed by your organization take precedence over everything
> - CLI flags like `--permission-mode` or `--settings` override `settings.json` for that session
> - Some environment variables take precedence over their equivalent setting, but this varies: check the **environment variables reference** for each one
>
> See **settings precedence** for the full order.

Click a filename to open that node in the explorer above.

File	Scope	Commit	What it does	Reference
`CLAUDE.md`	Project and global	✓	Instructions loaded every session	**Memory**
`rules/*.md`	Project and global	✓	Topic-scoped instructions, optionally path-gated	**Rules**
`settings.json`	Project and global	✓	Permissions, hooks, env vars, model defaults	**Settings**
`settings.local.json`	Project only		Your personal overrides, gitignored when Claude Code creates it	**Settings scopes**
`.mcp.json`	Project only	✓	Team-shared MCP servers	**MCP scopes**
`.worktreeinclude`	Project only	✓	Gitignored files to copy into new worktrees	**Worktrees**
`skills/<name>/SKILL.md`	Project and global	✓	Reusable prompts invoked with `/name` or auto-invoked	**Skills**
`commands/*.md`	Project and global	✓	Single-file prompts; same mechanism as	**Skills**

File	Scope	Commit	What it does	Reference
			skills	
`output-styles/*.md`	Project and global	✓	Custom system-prompt sections	**Output styles**
`agents/*.md`	Project and global	✓	Subagent definitions with their own prompt and tools	**Subagents**
`workflows/*.js`	Project and global	✓	Dynamic workflow scripts written by Claude and saved from `/workflows`; each file becomes a `/<name>` command	**Dynamic workflows**
`agent-memory/<name>/`	Project and global	✓	Persistent memory for subagents	**Persistent memory**
`~/.claude.json`	Global only		App state, OAuth, UI toggles, personal MCP servers	**Global config**
`projects/<project>/memory/`	Global only		Auto memory: Claude's notes to itself across sessions	**Auto memory**
`keybindings.json`	Global only		Custom keyboard shortcuts	**Keybindings**
`themes/*.json`	Global only		Custom color themes	**Custom themes**

Troubleshoot configuration

If a setting, hook, or file isn't taking effect, see **Debug your configuration** for the inspection commands and a symptom-first lookup table.

Application data

Beyond the config you author, `~/.claude` holds data Claude Code writes during sessions. These files are plaintext. Anything that passes through a tool lands in a transcript on disk: file contents, command output, pasted text.

Cleaned up automatically

Files in the paths below are deleted on startup once they're older than `cleanupPeriodDays`. The default is 30 days.

Path under `~/.claude/`	Contents
`projects/<project>/<session>.jsonl`	Full conversation transcript: every message, tool call, and tool result
`projects/<project>/<session>/subagents/`	**Subagent** conversation transcripts, removed with the parent session transcript when it ages out
`projects/<project>/<session>/tool-results/`	Large tool outputs spilled to separate files
`file-history/<session>/`	Pre-edit snapshots of files Claude changed, used for **checkpoint restore**
`plans/`	Plan files written during **plan mode**
`debug/`	Per-session debug logs, written only when you start with `--debug` or run `/debug`
`paste-cache/`, `image-cache/`	Contents of large pastes and attached images
`session-env/`	Per-session environment metadata
`tasks/`	Per-session task lists written by the task tools
`shell-snapshots/`	Captured shell environment used by the Bash tool. Removed on clean exit. The sweep clears any left after a crash.
`backups/`	Timestamped copies of `~/.claude.json` taken before config migrations
`feedback-bundles/`	Redacted transcript archives written by `/feedback` on third-party providers, for sending to your Anthropic account team
`todos/`, `statsig/`, `logs/`	Legacy directories from older versions. No longer written. The sweep removes their contents and then the empty directory.

Kept until you delete them

The following paths are not covered by automatic cleanup and persist indefinitely.

Path under `~/.claude/`	Contents
`history.jsonl`	Every prompt you've typed, with timestamp and project path. Used for up-arrow recall.
`stats-cache.json`	Aggregated token and cost counts shown by `/usage`

Path under `~/.claude/`	Contents
`remote-settings.json`	Cached copy of **server-managed settings** for your organization. Only present when your organization has configured them. Refreshed on each launch.

Other small cache and lock files appear depending on which features you use and are safe to delete.

Plaintext storage

Transcripts and history are not encrypted at rest. OS file permissions are the only protection. If a tool reads a `.env` file or a command prints a credential, that value is written to `projects/<project>/<session>.jsonl`. To reduce exposure:

- Lower `cleanupPeriodDays` to shorten how long transcripts are kept
- Set the `CLAUDE_CODE_SKIP_PROMPT_HISTORY` environment variable to skip writing transcripts and prompt history in any mode. In non-interactive mode, you can instead pass `--no-session-persistence` alongside `-p`, or set `persistSession: false` in the Agent SDK.
- Use **permission rules** to deny reads of credential files

Clear local data

Run `claude project purge` to delete the state Claude Code holds for one project. The command requires Claude Code v2.1.124 or later. It deletes:

- Transcripts and auto memory under `projects/`
- Per-session `tasks/`, `debug/`, and `file-history/` entries
- Matching prompt lines in `history.jsonl`
- The project's entry in `~/.claude.json`

The command prints the full deletion plan and asks for confirmation before removing anything.

Preview the plan without deleting anything:

```
claude project purge ~/work/my-repo --dry-run
```

Delete with a single confirmation prompt:

```
claude project purge ~/work/my-repo
```

Omit the path to pick a project from an interactive list.

Skip the confirmation prompt for use in scripts:

```
claude project purge ~/work/my-repo --yes
```

Pass `--all` instead of a path to purge state for every project at once, which deletes `history.jsonl` outright rather than filtering it. Pass `-i` to step through the deletion plan one item at a time.

The command leaves `shell-snapshots/` and `backups/` alone because those are not project-scoped, and warns about them in the plan output. It exits with status 1 if no state matches the given path.

You can also delete any of the application-data paths above by hand. New sessions are unaffected. The table below shows what you lose for past sessions.

Delete	You lose
`~/.claude/projects/`	Resume, continue, and rewind for past sessions
`~/.claude/history.jsonl`	Up-arrow prompt recall
`~/.claude/file-history/`	Checkpoint restore for past sessions
`~/.claude/stats-cache.json`	Historical totals shown by `/usage`
`~/.claude/remote-settings.json`	Nothing. Re-fetched on next launch.
`~/.claude/debug/`, `~/.claude/plans/`, `~/.claude/paste-cache/`, `~/.claude/image-cache/`, `~/.claude/session-env/`, `~/.claude/tasks/`, `~/.claude/shell-snapshots/`, `~/.claude/backups/`	Nothing user-facing
`~/.claude/todos/`, `~/.claude/statsig/`, `~/.claude/logs/`	Nothing. Legacy directories not written by current versions.

Don't delete `~/.claude.json`, `~/.claude/settings.json`, or `~/.claude/plugins/`: those hold

your auth, preferences, and installed plugins.

Related resources

- **Manage Claude's memory**: write and organize CLAUDE.md, rules, and auto memory
- **Configure settings**: set permissions, hooks, environment variables, and model defaults
- **Create skills**: build reusable prompts and workflows
- **Configure subagents**: define specialized agents with their own context